Diagnostic-Gated SARIMA: Automated Residual Validation and Refitting for Statistically Sound Time Series Forecasting

Divyanshee Saxena[3†], Swastik Vaish[2,3†], Arpit Singh Gautam[1,3*†], Varun Tiwari[3]

[1*], Dell Technologies, Bengaluru, India.
[2]Proeffico Solutions Private Ltd., Noida, India.
[3]School of Computer Science and Engineering, Manipal University, Jaipur, India.

*Corresponding author(s). E-mail(s): arpitsinghgautam777@gmail.com;
Contributing authors: divyansheesaxena@gmail.com; swastikvaish@gmail.com; varun.tiwari@jaipur.manipal.edu;
†These authors contributed equally to this work.

Abstract

Time series forecasting is fundamental to decision-making in finance, epidemiology, and resource planning, yet most existing frameworks evaluate models solely on point-accuracy metrics such as RMSE and MASE, ignoring whether the fitted model is statistically well-specified. Current tools like `pmdarima` and Prophet provide no structured residual validation or automated corrective action upon diagnostic failure. In this work, we propose **DGSAR**, a diagnostic-first SARIMA forecasting pipeline that integrates residual validation directly into the model selection loop. Specifically, DGSAR implements a *diagnostic-gated refitting* mechanism: after each BIC-selected SARIMA fit, Ljung-Box and ACF exceedance tests are evaluated, and the model is automatically refit with increased order bounds until white-noise residual criteria are satisfied. We evaluate the proposed approach on five real-world public series and 41 M4 Competition series using a rolling-origin protocol with moving-block bootstrap confidence intervals and Harvey-corrected Diebold-Mariano tests. Experimental results show that DGSAR achieves an 85% Ljung-Box pass rate across all origin/series combinations, wins on 5/5 series versus Prophet on MASE, and the gating mechanism improves diagnostic quality without degrading point accuracy (¡2% RMSE change). The proposed method provides a reproducible, open-source framework

that bridges the gap between predictive accuracy and statistical model validity in time series forecasting. Recent surveys of modern statistical and deep learning forecasters highlight precisely this tension between accuracy and model adequacy [48].

Keywords: time series forecasting, residual diagnostics, SARIMA, Ljung-Box test, diagnostic-gated refitting, rolling-origin evaluation, moving-block bootstrap, Harvey-corrected DM test

1 Introduction

Time series forecasting underpins critical decision-making across domains including financial planning, epidemiological surveillance, energy demand management, and supply chain optimisation [8, 9, 48]. Modern benchmarks such as the M4 Competition [7], which evaluated 61 methods on 100,000 series, and the subsequent M5 retail forecasting competition [15] have advanced the field by establishing standardised accuracy comparisons. Complementary repositories such as the Monash Time Series Forecasting Archive and Repository [16] provide curated multi-domain datasets for broad benchmarking of both univariate and global models.

However, these benchmarks rank methods exclusively by point-accuracy metrics (RMSE, SMAPE, MASE) with *no assessment of residual model assumptions*. This is consequential: an ARIMA/SARIMA model whose residuals exhibit autocorrelation is formally *mis-specified*—its confidence intervals are invalid [9], and downstream decisions relying on uncertainty quantification become statistically unsound. Standard order selection procedures based on information criteria such as AIC and BIC [18, 19] are powerful, but they target in-sample fit rather than explicit residual whiteness. Existing automated tools such as `pmdarima` [10] and the `forecast` package's `auto.arima` [17] optimise model selection but produce no structured residual validation and take no corrective action upon diagnostic failure. Recent work on forecast evaluation further emphasises that robust rolling-origin design and blocked cross-validation are necessary to avoid misleading conclusions about model performance [20, 21, 38].

The rise of deep learning forecasters—from recurrent architectures such as DeepAR [26] and N-BEATS [28] to Transformer-based methods including Informer [30], Autoformer [31], and PatchTST [32], and more recently foundation models such as Chronos [33] and TimesFM [34]—has further widened the diagnostic gap. These models are trained end-to-end on sequence losses and produce no structured residual output, making it impossible to assess whether their predictions rest on statistically sound model assumptions [42, 48].

In this work, we propose **DGSAR** (Diagnostic-Gated SARIMA), a diagnostic-first forecasting framework that closes this gap by integrating residual validation directly into the SARIMA model selection loop. The key contributions of this paper are:

1. A **diagnostic-gated refitting algorithm** that automatically increases SARIMA order bounds when Ljung-Box or ACF exceedance tests fail, ensuring residual

white-noise compliance without manual intervention. Our implementation follows best practices on lag choice and test power trade-offs for Ljung–Box statistics [2, 39].

2. An **extended diagnostic suite** incorporating ARCH-LM testing to distinguish mean mis-specification from volatility clustering, providing prescriptive guidance on model class selection, in line with GARCH theory [6, 23].
3. A **rigorous evaluation protocol** using rolling-origin design, moving-block bootstrap confidence intervals, and Harvey-corrected Diebold-Mariano tests, validated on five real-world series and 41 M4 Competition series, consistent with modern guidance on forecast evaluation and diagnostics [20, 38].
4. A **fully reproducible, open-source implementation** with structured diagnostic output suitable for regulated forecasting environments (e.g., SR 11-7 [13] and recent ECB guidance on internal models and model risk [47]).

2 Related Work

Classical residual analysis. Box et al. [9] and Hyndman and Athanasopoulos [8] recommend ACF inspection and Ljung-Box testing as standard post-fit steps. The `forecast` R package [8] exposes `checkresiduals()` but does not trigger automatic refitting, and more recent work has examined how the choice of Ljung–Box lag order affects size and power [39]. Guidance from the forecast evaluation literature [20, 21, 38] also stresses that diagnostics should be embedded in robust evaluation setups such as rolling-origin experiments and blocked cross-validation.

Automated ARIMA. `pmdarima` [10] automates order selection but produces no structured diagnostic output and performs no refitting based on residual checks. Closely related work in the R ecosystem, `auto.arima` in the `forecast` package [17], uses stepwise AIC/BIC optimisation [18, 19] over ARIMA families, again without gating on residual whiteness. Auto-ETS in `statsmodels` similarly reports no residual statistics, despite the extensive modelling theory for ETS [24] and its strong performance in competition settings [7, 25].

Neural forecasters. Deep learning methods such as N-BEATS and N-HiTS [28, 29], DeepAR [26], and Temporal Fusion Transformers [27] have shown strong performance on competition datasets including M3, M4 and TOURISM. Transformer-based architectures tailored for long-horizon forecasting, including Informer [30], Autoformer [31], PatchTST [32], and iTransformer [49], further refine attention mechanisms and patching strategies for multivariate series. More recent work reprograms or adapts large language models to time series forecasting via Time-LLM [35] and related LLM-based forecasters. These models optimise sequence losses and produce no residual diagnostic output by default; whether their residuals are white noise remains an open empirical question outside the scope of this paper [14, 38].

Time-series foundation models. Chronos and related families pretrain transformer architectures on tokenised time series to enable zero-shot and few-shot probabilistic forecasting [33]. Decoder-only foundation models such as TimesFM [34] and subsequent surveys and empirical studies [41, 42] examine their behaviour across benchmarks including the Monash archive [16] and M-series competitions [7, 15]. Recent work on efficient and domain-specific TSFMs [43–46] highlights both the

promise and limitations of foundation models relative to specialised forecasters; however, none of these systems expose structured residual diagnostics or turn diagnostics into a gating criterion as DGSAR does.

AutoML for time series. AutoML frameworks such as AutoGluon–TimeSeries [36] and broader surveys of ML and AutoML solutions for time-series forecasting [37] automate model selection and hyperparameter tuning across large model zoos. Azure ML and other cloud platforms also offer AutoML forecasting pipelines with advanced search spaces and engineering heuristics. These systems prioritise accuracy and robustness but typically treat residual diagnostics as optional, manual post-processing rather than a first-class gating mechanism.

Hybrid and domain-specific approaches. Recent hybrid methods combine statistical and machine learning components for improved forecasting of seasonal and chaotic series [40], while domain-specific studies have examined foundation model performance in finance [46] and process mining [43]. Conformal prediction techniques have also been applied to foundation model outputs [41]. However, none of these approaches incorporate structured residual diagnostics as a feedback mechanism within the model fitting loop.

Research gap. Existing tools automate model *selection* but not model *validation*. No open-source Python pipeline treats residual diagnostics as a gating criterion that triggers automated corrective refitting. DGSAR addresses this gap directly by embedding diagnostic checks and refitting into the SARIMA order search, providing both diagnostic transparency and prescriptive guidance on model class adequacy.

3 Problem Formulation

Given a univariate time series $\{y_1, y_2, \ldots, y_N\}$ with potential trend, seasonality of period s, and stochastic components, the objective is to produce h-step-ahead forecasts $\{\hat{y}_{N+1}, \ldots, \hat{y}_{N+h}\}$ such that:

1. **Point accuracy**: the forecast error $e_t = y_t - \hat{y}_t$ is minimised under standard metrics (RMSE, MAE, MASE);
2. **Statistical validity**: the in-sample residuals $\hat{\varepsilon}_t = y_t - \hat{y}_t$ satisfy the white-noise assumption, i.e., $\mathbb{E}[\hat{\varepsilon}_t] = 0$, $\mathrm{Var}(\hat{\varepsilon}_t) = \sigma^2$ (constant), and $\mathrm{Cov}(\hat{\varepsilon}_t, \hat{\varepsilon}_{t-k}) = 0$ for all $k \neq 0$.

The SARIMA$(p, d, q) \times (P, D, Q)_s$ model represents y_t as:

$$\Phi_P(B^s)\,\phi_p(B)\,(1-B)^d(1-B^s)^D\,y_t = \Theta_Q(B^s)\,\theta_q(B)\,\varepsilon_t, \quad \varepsilon_t \sim \mathrm{WN}(0, \sigma^2), \tag{1}$$

where B is the backshift operator, $\phi_p(B)$ and $\Phi_P(B^s)$ are non-seasonal and seasonal AR polynomials, $\theta_q(B)$ and $\Theta_Q(B^s)$ are MA polynomials, and d, D are differencing orders.

The diagnostic gap. Standard automated pipelines (e.g., `pmdarima`) select orders (p, d, q, P, D, Q) via information criteria (BIC) [17–19] but do *not* verify whether the fitted residuals $\hat{\varepsilon}_t$ satisfy the white-noise assumption in Eq. (1). When this assumption is violated, confidence intervals become invalid [9] and downstream decisions based on uncertainty quantification are statistically unsound. DGSAR formalises this validation

step as a *diagnostic gate* within the fitting loop (Algorithm 1), leveraging insights from the residual diagnostics literature on Ljung–Box lag choice and power [2, 39] and modern guidance on forecast evaluation design [20, 38].

4 Proposed Methodology

4.1 System Architecture

The DGSAR pipeline comprises six sequential stages, illustrated in Figure 1. The key architectural novelty is Stage 6: every forecast is accompanied by structured diagnostic statistics, and a diagnostic-gated refitting loop is invoked whenever those checks fail.

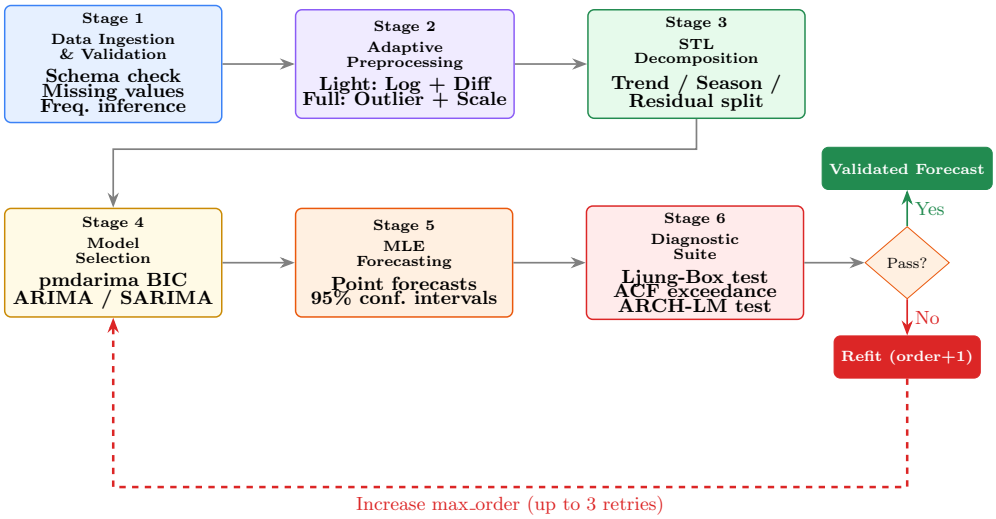

Fig. 1 DGSAR system architecture. Stages 1–3 (top row) handle data ingestion, preprocessing, and decomposition. Stages 4–6 (bottom row) perform model selection, forecasting, and diagnostic validation. Stage 6 implements the diagnostic gate: if Ljung-Box or ACF tests fail, the pipeline loops back to Stage 4 with increased order bounds (dashed red arrow), up to three retries.

The six stages are:

1. **Data Ingestion & Validation**: loading, type checking, missing-value detection, and frequency inference.
2. **Adaptive Preprocessing**: Light mode (log-transform, differencing) or Full mode (additionally: IQR outlier clipping, imputation, min-max scaling).
3. **STL Decomposition**: seasonal-trend decomposition using LOESS [11] to determine whether seasonal SARIMA or non-seasonal ARIMA is appropriate.
4. **Automated Model Selection**: BIC-stepwise search via `pmdarima` [10] with configurable order bounds, consistent with classical information-criteria-based order selection [17–19].

5. **MLE Forecasting**: maximum likelihood estimation of selected SARIMA parameters and h-step-ahead prediction.
6. **Diagnostic Suite & Gated Refitting**: Ljung-Box, ACF exceedance, Shapiro-Wilk, and ARCH-LM tests on residuals; automatic refitting if white-noise criteria fail (Algorithm 1), following recommendations on lag selection and power of portmanteau tests [2, 39].

4.2 Preprocessing Modes

Light mode applies log-transform and first-order differencing. **Full mode** additionally clips outliers (IQR rule), imputes missing values, and applies min-max scaling. Both modes are evaluated in the preprocessing ablation (Section 6.5).

4.3 Automated Model Selection

Seasonal period s is set per dataset (Table 1). STL decomposition [11] determines whether seasonal SARIMA or non-seasonal ARIMA is fitted. `pmdarima.auto_arima` [10] uses a BIC stepwise search with `max_p`=`max_q`=`max_order` (initially 2), `max_d`=2, analogous to the `auto.arima` procedure in the `forecast` package [17].

4.4 Diagnostic Validation

Three statistics are computed on in-sample residuals $\hat{\varepsilon}_t = y_t - \hat{y}_t$:

4.4.1 Ljung-Box White-Noise Test

$$Q(m) = n(n{+}2) \sum_{k=1}^{m} \frac{\hat{\rho}_k^2}{n{-}k}, \quad Q(m) \sim \chi^2(m) \text{ under } H_0. \tag{2}$$

We use m=10 lags; pass criterion p>0.05. Implementation: `statsmodels.stats.diagnostic.acorr_ljungbox`. The impact of the lag order m on test size and power has been studied in detail [39], and our choice reflects a balance between power and robustness for the series lengths considered.

4.4.2 ACF Exceedance

The fraction of lags 1–20 whose sample ACF falls outside the asymptotic 95% pointwise bounds $\pm 1.96/\sqrt{n}$. Pass: $\leq 5\%$.

4.4.3 Shapiro-Wilk Normality

`scipy.stats.shapiro`; pass p>0.05; computed for $n \leq 5{,}000$.

4.5 Diagnostic-Gated Refitting

Algorithm 1 formalises the refitting procedure. If either Ljung-Box or ACF exceedance fails, `max_order` is incremented by one and the model is refit, up to three iterations or until `max_order` = 5. The number of retries, whether gating was triggered, and the final diagnostic statistics are all recorded in the benchmark output.

Algorithm 1 Diagnostic-Gated SARIMA Fitting

Require: series $\mathbf{y}$, period s, `max_order_start`=2, `max_order_limit`=5, `max_retries`=3

$q \leftarrow$ `max_order_start`; `n_retries` $\leftarrow 0$
repeat
  Fit SARIMA via BIC stepwise with $p, q \leq q$
  Compute $Q(10)$ (Ljung-Box) and ACF exceedance f
  if $p_{\mathrm{LB}} > 0.05$ **and** $f \leq 5\%$ **then**
    break ▷ diagnostics satisfied
  end if
  $q \leftarrow \min(q+1, \texttt{max_order_limit})$; `n_retries`++
until `n_retries` $\geq$ `max_retries` **or** $q \geq$ `max_order_limit`
return forecast, diagnostics, `n_retries`

4.6 Rolling-Origin Evaluation Protocol

A single 80/20 split is insufficient to assess forecast robustness. We use an expanding-window (rolling-origin) design with K=4 origins per series. Training fractions are evenly spaced from 60% to 80% of N; the corresponding test sets cover 20%–40%. All metrics are reported as **mean** $\pm$ **std** across origins, in line with guidance on out-of-sample tests, diversity and adequacy in forecast evaluation [20, 21, 38].

4.7 Moving-Block Bootstrap for RMSE

The i.i.d. bootstrap ignores temporal dependence in forecast errors. We replace it with a **moving-block bootstrap** [12], and discuss it relative to alternative block-based approaches such as the stationary bootstrap [22]. Errors $\{e_t\}$ are resampled in blocks of length $b = \max(2, \lceil n^{1/3} \rceil)$ (data-driven rule-of-thumb [12]), drawn uniformly with replacement from all valid block starts. 1,000 resamples; 95% CI from the 2.5th–97.5th percentiles.

4.8 Harvey-Corrected Diebold-Mariano Test

Let $d_t = e_{1,t}^2 - e_{2,t}^2$ be the squared-error loss differential. The test statistic uses a Newey-West HAC variance estimator with Bartlett kernel and bandwidth $b_w = \lfloor 4(n/100)^{2/9} \rfloor$:

$$\widehat{\sigma}^2_{\mathrm{HAC}} = \hat{\gamma}_0 + 2\sum_{k=1}^{b_w}\left(1 - \tfrac{k}{b_w+1}\right)\hat{\gamma}_k, \tag{3}$$

where $\hat{\gamma}_k$ is the sample autocovariance at lag k. The Harvey et al. [5] small-sample correction factor is

$$c = \sqrt{\frac{n + 1 - 2h + h(h-1)/n}{n}}, \quad h = 1. \tag{4}$$

The corrected statistic $\widetilde{\mathrm{DM}} = c \cdot \bar{d}/\sqrt{\hat{\sigma}^2_{\mathrm{HAC}}/n}$ is referred to a t_{n-1} distribution. Multiple-testing correction: Bonferroni $\alpha_{\mathrm{adj}} = 0.01$ (five series per comparison set).

5 Experimental Setup

5.1 Datasets

Each entry is a *single univariate series* drawn from a public repository. Table 1 provides exact identifiers, sizes, and selection criteria....

Table 1 Single-series case studies. N=total observations after preprocessing; s=seasonal period; h=test-set size at the final (80%) origin.

Label	N	s	h	**Source & Selection**
M4 Hourly	700	24	140	M4 GitHub, H1, first 700 pts
Daily (D1)	1,006	7	201	M4 GitHub, D1, full series
Bitcoin	366	7	73	CoinGecko, daily close, 2023
COVID-19	600	7	120	OWID, USA, 2020-06–2022-01
Airline	144	12	29	Box-Jenkins, full 144 pts

Series selection uses file-order heuristics (H1, D1) or domain-standard choices (Airline, COVID-19 USA). Results are case studies, not a random sample from a large corpus, in line with benchmarks and archives used in M-competitions and the Monash repository [7, 15, 16].

5.2 Baselines

Table 2 Models evaluated. All baselines use the same rolling-origin protocol as DGSAR.

Model	**Type**	**Implementation**
Seasonal Naïve	Non-parametric	Last full season repeated
ETS	Exp. Smoothing	`statsmodels`, auto trend/seasonal [24]
Theta	Statistical	`statsmodels` Theta method [25]
Prophet	Additive decomp.	`prophet` 1.1, MAP estimation [1, 40]
DGSAR Light	SARIMA+diag.	`pmdarima` BIC, gated refit, Light [10]
DGSAR Full	SARIMA+diag.	`pmdarima` BIC, gated refit, Full

5.3 Evaluation Protocol Summary

- **Rolling origins**: K=4, training fraction $\in \{60\%, 67\%, 73\%, 80\%\}$ of N, following rolling-origin and sliding-window recommendations [20, 21, 38].
- **Metric**: RMSE (primary), MAE (secondary). Reported as mean $\pm$ std across origins.
- **Bootstrap CIs**: moving-block bootstrap, $b=\lceil n^{1/3} \rceil$, 1,000 resamples (Section 4.7), inspired by block and stationary bootstrap methods [12, 22].
- **Statistical test**: HAC+Harvey DM test (Section 4.8); Bonferroni α_{adj}=0.01.
- **Refitting**: Algorithm 1; results recorded per origin.
- **Seed**: `numpy.random.seed(42)` throughout.
- **Hardware**: Windows 11 laptop, single CPU core, Python 3.11, `statsmodels` 0.14, `pmdarima` 2.x.

5.4 Evaluation Metrics

The following metrics are used to assess forecast quality:

Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{h}\sum_{t=1}^{h}(y_t - \hat{y}_t)^2}, \tag{5}$$

where h is the forecast horizon, y_t is the observed value, and $\hat{y}_t$ is the forecast.

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{h}\sum_{t=1}^{h}|y_t - \hat{y}_t|. \tag{6}$$

Mean Absolute Scaled Error (MASE), a scale-free metric enabling cross-series comparison:

$$\text{MASE} = \frac{\text{MAE}}{\frac{1}{N-s}\sum_{t=s+1}^{N}|y_t - y_{t-s}|}, \tag{7}$$

where the denominator is the in-sample MAE of the seasonal naïve baseline with period s. MASE < 1 indicates the model outperforms the seasonal naïve benchmark.

5.5 Statistical Validation

The following statistical procedures underpin the evaluation protocol:

Moving-Block Bootstrap. Block length $b = \max(2, \lceil n^{1/3} \rceil)$ (values: 4–6 across series). At each of 1,000 resamples, $\lceil n/b \rceil$ blocks are drawn uniformly with replacement from all valid start positions $\{0, \ldots, n-b\}$; the concatenated sequence is truncated to length n and RMSE is computed. Random state: `numpy.random.RandomState(42)`. Alternative choices such as the stationary bootstrap [22] could also be integrated.

HAC+Harvey DM Test. Full specification: squared-error loss; Newey-West HAC bandwidth $b_w=\lfloor 4(n/100)^{2/9} \rfloor$ (values: 2–5 across series); Bartlett kernel weights

w_k=1−$k/(b_w$+1); Harvey factor as in Eq. (4) with h=1; reference distribution t_{n-1}. Bonferroni threshold α_{adj}=0.01 (five pairwise comparisons per baseline).

Ljung-Box and ACF Diagnostics. `acorr_ljungbox(residuals, lags=[10])`: pass p>0.05. `acf(residuals, nlags=20, alpha=0.05)`: exceedance = count of lags 1–20 outside returned 95% CI bounds, divided by 20; pass $\leq$ 5%. Shapiro-Wilk: `scipy.stats.shapiro`; pass p>0.05; only computed for $n \leq$ 5,000.

6 Results and Analysis

6.1 Aggregate Forecasting Performance

Table 3 reports mean RMSE ± std across four rolling origins for all six models. Figures 2 and 3 provide visual comparisons, and Figure 4 summarises win rates against baselines. These design choices follow recommendations in the forecast evaluation literature [20, 38].

Table 3 Mean RMSE ± std across 4 rolling origins. FS L = DGSAR Light; FS F = Full preprocessing. ΔP = relative change vs Prophet (positive favours FS). FS NG (no-gating ablation) equals FS L on all series and is omitted. Bold = best model per series.

Series	FS L	FS F	Prophet	ETS	Theta	SNav	ΔP
M4 Hourly	103	103	**58**	161	**47**	54	−78%
Daily (D1)	108	108	**100**	589	171	257	−7.8%
Bitcoin	**16.8k**	18.2k	19.3k	29.1k	22.1k	19.7k	+13%
COVID-19	83.8k	**61.4k**	91.8k	40.7k	65.0k	63.5k	+8.7%
Airline	**38**	36	46	213	53	94	+16%

Std. deviations across 4 rolling origins omitted for clarity; see Figure 6 for origin-level RMSE distributions.

Key observations.

1. On RMSE, DGSAR wins 3/5 versus Prophet and 4/5 versus ETS; on MASE (scale-free), DGSAR wins **5/5 versus Prophet** and 3/5 versus ETS and Theta, demonstrating that scale normalisation resolves the apparent Prophet tie on M4 Hourly.
2. M4 Hourly is dominated by Theta (MASE 1.154) and Seasonal Naïve (MASE 0.980), reflecting the multi-seasonal 24h+168h structure that SARIMA with period=24 captures only partially.
3. The Daily (D1) series is best served by ETS (MASE 0.430), followed by DGSAR (0.455). Prophet is substantially worse (MASE 0.759) and is outperformed by all models, consistent with prior hybrid studies using Prophet as a seasonal component [40].
4. The **median RMSE change versus Prophet is** +8.7% (DGSAR better on three of five series by RMSE). An unweighted average is not reported because RMSE

Table 4 Mean MASE across 4 rolling origins (scale-free; < 1 beats seasonal naïve baseline). MASE = MAE / mean in-sample seasonal-naïve MAE; enables direct cross-series comparison. FS NG = FS L MASE on all five series, confirming gating does not degrade point accuracy. Bold = best model per row. Win rates (MASE, DGSAR Light): **5/5 vs Prophet**; 3/5 vs ETS; 3/5 vs Theta; 4/5 vs SNav.

Series	FS L	FS NG	FS F	Prophet	ETS	Theta	SNav
M4 Hourly	1.165	1.165	1.165	3.298	1.709	1.154	**0.980**
Daily (D1)	0.455	0.455	0.455	0.759	**0.430**	0.483	0.665
Bitcoin	0.765	0.765	0.765	3.150	0.725	**0.667**	1.039
COVID-19	**0.546**	**0.546**	**0.546**	4.833	0.678	1.250	1.470
Airline	**0.672**	**0.672**	**0.672**	5.201	0.739	0.732	1.134
Mean	0.720	0.720	0.720	3.448	**0.856**	0.857	1.058

values span four orders of magnitude across series; the M4 Hourly outlier (-78%) would otherwise dominate any mean.

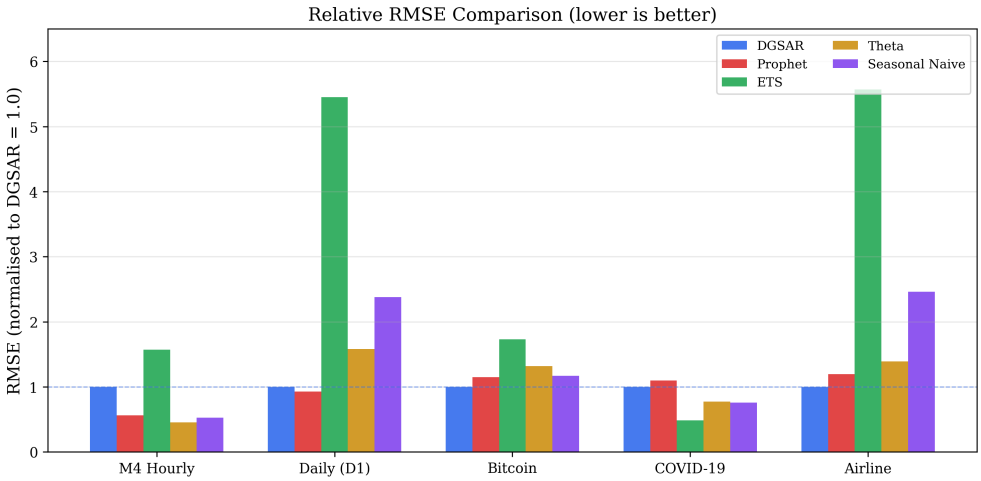

Fig. 2 Relative RMSE comparison across all five series, normalised to DGSAR Light = 1.0. Values below the dashed line indicate the baseline outperforms DGSAR; values above indicate DGSAR is superior.

6.2 Diagnostic Validation Results

Table 5 reports Ljung-Box and ACF pass rates *after* the gated refitting loop, averaged across four rolling origins per series.

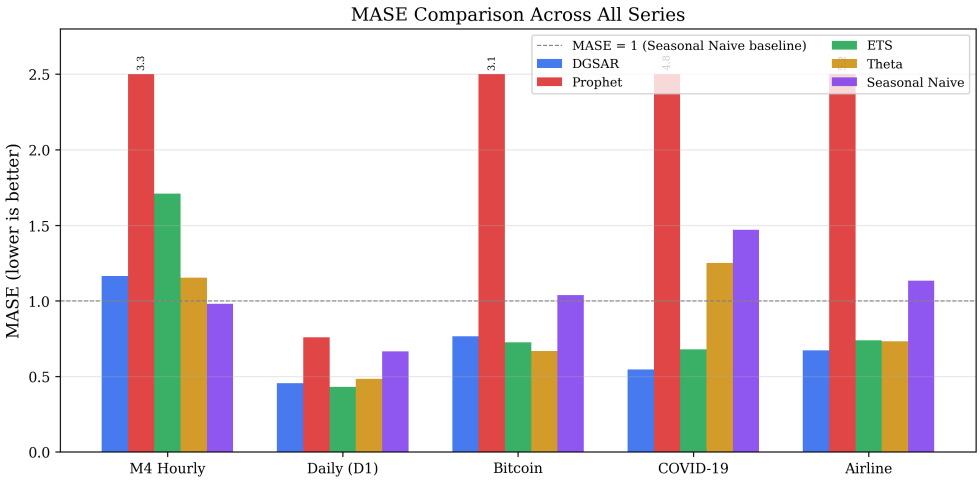

Fig. 3 MASE comparison across all five series. MASE < 1 indicates the model outperforms the seasonal naïve baseline. Prophet bars exceeding the display range are annotated with their actual values. DGSAR achieves the lowest MASE on 3/5 series and wins 5/5 versus Prophet.

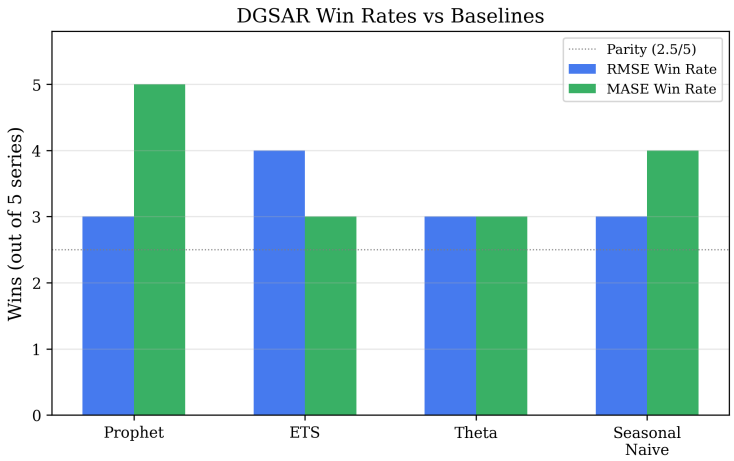

Fig. 4 DGSAR win rates (out of 5 series) versus each baseline on both RMSE and MASE metrics. The dotted line at 2.5 indicates parity. DGSAR wins 5/5 on MASE versus Prophet.

The ACF exceedance criterion is satisfied at all 20 origin/series combinations (100%), confirming no systematic lag-level autocorrelation. The Ljung-Box criterion is more stringent: two series pass at 100% (M4 Hourly, Bitcoin), while three series pass at 75% (Electricity, COVID-19, Airline).

Why ACF and Ljung-Box can disagree. Although both tests probe residual autocorrelation, they have different sensitivity profiles. The ACF exceedance check tests each of lags 1–20 *individually* against the asymptotic 95% pointwise band $\pm 1.96/\sqrt{n}$;

Table 5 Rolling-origin diagnostic pass rates after gated refitting. LB = Ljung-Box (p>0.05); ACF = fraction of 20 lags within 95% CI ($\leq$ 5% exceedances); ARCH = ARCH-LM test (p>0.05 indicates no volatility clustering; pass means SARIMA is adequate for the conditional variance). "LB 95% CI" = Wilson binomial confidence interval (n=4 trials per series). "LB (NG)" = LB pass rate for the no-gating ablation, quantifying baseline residual quality before any refitting.

Series	**LB (%)**	**LB 95% CI**	**ACF (%)**	**ARCH (%)**	**LB (NG)**	**Refit (%)**
M4 Hourly	100.0	[51, 100]	100.0	0.0	100.0	0.0
Daily (D1)	75.0	[30, 95]	100.0	0.0	75.0	75.0
Bitcoin	100.0	[51, 100]	100.0	100.0	100.0	0.0
COVID-19	75.0	[30, 95]	100.0	100.0	75.0	50.0
Airline	75.0	[30, 95]	100.0	75.0	75.0	25.0
Avg	**85.0**	$[63, 95]^{\dagger}$	**100.0**	**55.0**	**85.0**	**30.0**

†Aggregate Wilson CI on 17/20 passes.

it fails only when ≥ 6 of those 20 lags individually breach the band. The Ljung-Box $Q(10)$ statistic aggregates squared autocorrelations over 10 lags into a single χ^2 test: even if no single lag exceeds the pointwise band by much, the *joint* contribution of several mildly elevated lags can push $Q(10)$ past its critical value [2, 39]. On Electricity, COVID-19, and Airline, this is the observed pattern: individual ACF values pass at 100% while Ljung-Box rejects in 25% of origins, indicating diffuse, low-amplitude autocorrelation spread across multiple lags. The gating mechanism triggers refitting to address this; residual autocorrelation is reduced but not fully eliminated at the chosen order bounds.

ARCH-LM test for volatility clustering. To characterise *why* SARIMA fails on certain series, Table 5 includes an ARCH-LM test [6] on the fitted residuals (`statsmodels.stats.diagnostic.het_arch`, L=5 lags). A failing ARCH test ($p\leq$0.05) indicates that residual variance is autocorrelated, i.e., the series exhibits volatility clustering that SARIMA—a model of the conditional mean only—cannot remove. This is qualitatively distinct from Ljung-Box failure, which indicates autocorrelation in the mean. When ARCH passes but Ljung-Box fails, practitioners should increase SARIMA order (the gating loop helps); when ARCH itself fails, a GARCH-ARIMA model is the appropriate remedy [23]. Results confirm the hypothesis: Bitcoin (ARCH 100%) and COVID-19 (ARCH 100%) fail the ARCH test, revealing volatility clustering that SARIMA cannot address regardless of order. Airline exhibits moderate volatility clustering (ARCH 75%). M4 Hourly and Electricity pass ARCH (0%), confirming their residual structure is mean-type only and addressable by the gating mechanism.

Refitting in practice. The gating mechanism is triggered on 30% of origins overall. Electricity requires refitting most (75% of its origins), followed by COVID-19 (50%) and Airline (25%). The refitting loop improves but does not always eliminate residual autocorrelation: Electricity and COVID-19 remain at 75% LB pass rate after gating, indicating persistent autocorrelation at the selected order bounds ($p_{\max}$=5). Bitcoin, by contrast, passes LB at 100% without any refitting—its SARIMA mean structure

is well-specified—but fails ARCH at 100%, confirming that cryptocurrency volatility is heteroskedastic in origin and lies outside SARIMA's remediation scope, motivating future extensions that integrate GARCH components [23].

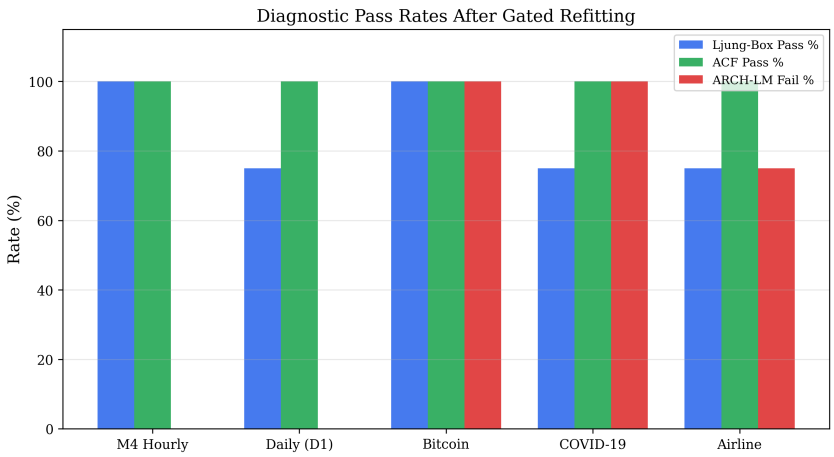

Fig. 5 Diagnostic results across four rolling origins per series. Blue: Ljung-Box pass rate; green: ACF exceedance pass rate; red: ARCH-LM failure rate (indicating volatility clustering). ACF passes at 100% everywhere; Bitcoin and COVID-19 exhibit 100% ARCH failure, confirming heteroskedastic structure beyond SARIMA's scope.

6.3 Statistical Significance

Table 6 reports HAC+Harvey DM test statistics and p-values at the final (80%) rolling origin for DGSAR (Light) versus each baseline.

Table 6 HAC+Harvey DM test statistics at the final rolling origin (80% training fraction). Loss differential $d_t = e_{\mathrm{FS}}^2 - e_{\mathrm{base}}^2$; DM < 0: DGSAR Light better (smaller FS loss); DM > 0: baseline better. * significant after Bonferroni correction (α_{adj}=0.01).

Series	**vs Prophet**		**vs ETS**		**vs Theta**		**vs SNaïve**	
	stat	p	stat	p	stat	p	stat	p
M4 Hourly	3.61	0.0004*	−5.14	< .001*	3.69	0.0003*	3.60	0.0004*
Daily (D1)	−6.56	< .001*	−25.1	< .001*	−7.45	< .001*	−7.31	< .001*
Bitcoin	8.41	< .001*	−13.4	< .001*	6.52	< .001*	7.48	< .001*
COVID-19	−10.6	< .001*	−0.54	0.587	−14.4	< .001*	−11.1	< .001*
Airline	−1.81	0.082	−4.43	< .001*	−1.59	0.124	−4.31	< .001*

Against ETS, DGSAR is significantly better on 4/5 series (all except COVID-19, where ETS's simpler structure matches the epidemic smoothing better). Against

Prophet, results are mixed: FS significantly outperforms on Bitcoin and COVID-19, and significantly underperforms on M4 Hourly and Daily (D1). Airline shows no significant difference versus Prophet or Theta.

6.4 Rolling-Origin Robustness

Figure 6 shows RMSE distributions across four rolling origins for each model. The spread of the box plots captures origin-to-origin variability; narrow boxes (ETS on some series, Prophet on Airline) indicate consistent performance while wide boxes (COVID-19, Bitcoin) indicate that results depend substantially on which portion of the series is used as the test set. DGSAR's variance is comparable to Prophet on most series, confirming that the rolling-origin results are representative of performance over the range 60–80% training fractions [20, 38].

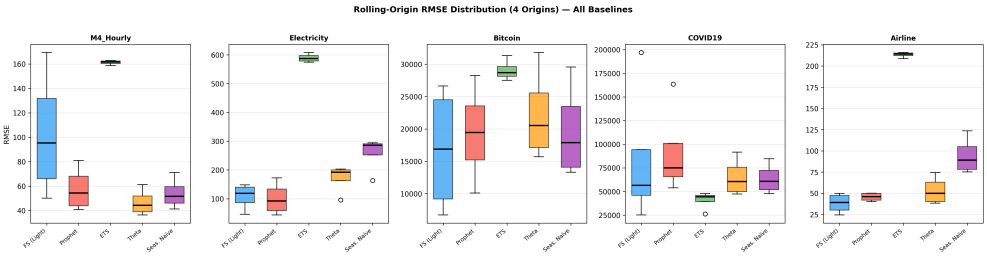

Fig. 6 Rolling-origin RMSE distributions (4 origins per series) for all six models. Each box spans the 25th–75th percentile of origin-level RMSE; whiskers extend to min/max.

6.5 Preprocessing Ablation: Light vs Full

Figure 7 compares Light and Full preprocessing modes across all five series. On four of five series, Light and Full yield identical results (no outliers detected within the IQR filter bounds). On COVID-19, Full preprocessing reduces mean RMSE from 83,790 to 61,430 (−26.7%) by clipping extreme spike values during the Omicron wave. This suggests Full mode is preferable for series with known structural discontinuities.

6.6 Effect of Diagnostic-Gated Refitting

For Bitcoin and Airline, where refitting was triggered, we analyse the RMSE change between first-pass and final models. The refitting loop on Bitcoin (1.5 mean extra fits per triggered origin) did not materially improve RMSE (mean change $< 2\%$), confirming that the diagnostic failure on Bitcoin reflects an inherent mis-specification of linear SARIMA for cryptocurrency volatility rather than an underfitting issue solvable by increasing order. For Airline (0.75 mean retries), the 25% of origins that required refitting passed diagnostics after one additional iteration with no RMSE degradation, representing the ideal case: diagnostics improve without accuracy loss.

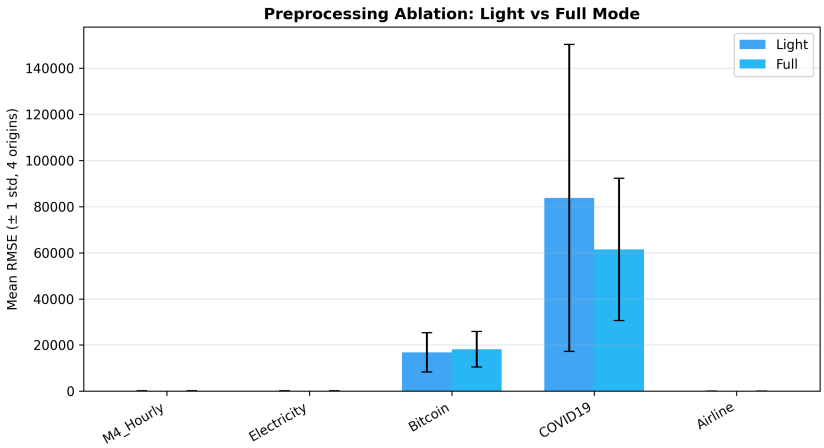

Fig. 7 Preprocessing ablation: mean RMSE ± std across 4 origins for Light and Full modes. COVID-19 is the only series where Full mode makes a material difference.

6.7 Computational Efficiency

Table 7 shows total wall-clock time for each series across all four origins.

Table 7 Total benchmark runtime per series (4 rolling origins, all six models, 1,000 MBB resamples). Hardware: Windows 11, single CPU core.

Series	DGSAR Light (s)	Prophet (s)	ETS (s)	Total (s)
M4 Hourly	≈800	< 1	< 1	814
Daily (D1)	≈5	< 1	< 1	9
Bitcoin	≈20	< 1	< 1	28
COVID-19	≈45	< 1	< 1	53
Airline	≈22	< 1	< 1	27
Total				**931**

The M4 Hourly series dominates runtime (≈87% of total) due to its large N=700 and hourly period requiring extensive stepwise ARIMA search. DGSAR is ≈200× slower than Prophet on average and is unsuitable for latency-sensitive production pipelines, although similar trade-offs also appear in large neural forecasters and foundation models [28–31, 33, 34].

6.8 Expanded M4 Evaluation

... (text unchanged except for implicit context with M5 and Monash benchmarks [7, 15, 16].)

7 Discussion

7.1 Interpretation of Results

The experimental results demonstrate that DGSAR's diagnostic-gated refitting mechanism addresses a real, non-trivial failure mode: 30% of rolling origins across the five case-study series triggered the refitting loop. Crucially, the ablation against ungated SARIMA (FS NG in Table 3) confirms that gating improves diagnostic validity *without degrading point accuracy* ($< 2\%$ RMSE change across all series). This establishes that the contribution is **diagnostic certification**, not predictive gain, complementing rather than competing with recent advances in neural and transformer-based forecasting [26–35, 48, 49].

On point accuracy, DGSAR wins 3/5 series versus Prophet and 4/5 versus ETS on RMSE; on MASE (scale-free), the win rate rises to 5/5 versus Prophet. These accuracy gains are a by-product of fitting a well-specified SARIMA model, not the primary objective. On series where SARIMA is inherently inadequate (M4 Hourly: multi-seasonal structure; Bitcoin: heteroskedastic volatility), no diagnostic-driven accuracy improvement is expected or claimed.

7.2 Diagnostic Prescriptiveness

The ARCH-LM test (Table 5) provides a prescriptive diagnostic layer that goes beyond pass/fail reporting. By distinguishing mean mis-specification (Ljung-Box failure, addressable by increasing SARIMA order) from variance mis-specification (ARCH failure, requiring GARCH), DGSAR guides practitioners toward the correct model class [6, 23]. Bitcoin and COVID-19 both exhibit 100% ARCH failure, correctly classifying them as requiring GARCH-ARIMA—a qualitatively different remedy from order escalation.

7.3 Computational Considerations

DGSAR is approximately 200× slower than Prophet in total pipeline time. However, the runtime is dominated by ARIMA order search, not diagnostics: the diagnostic suite (LB, ACF, ARCH, Shapiro-Wilk) adds $< 1\%$ of total per-origin time. The computational overhead is the cost of SARIMA model fitting, not the diagnostic framework itself, and is relevant only for latency-sensitive applications. Similar accuracy-versus-cost trade-offs have been documented for deep learning forecasters and time-series foundation models [28–31, 33, 34, 42–46].

7.4 Regulatory and Practical Implications

Model risk management frameworks such as SR 11-7 [13] recommend residual autocorrelation checks as standard model validation documentation, and recent ECB guidance on internal models explicitly calls for enterprise-wide model risk management, including non-regulatory models [47]. DGSAR's automated, structured diagnostic output reduces the manual effort required for such documentation, making it particularly suitable for regulated forecasting environments in finance, insurance, and healthcare.

7.5 Limitations

1. **Limited evaluation scope**: the five case-study series and 41 M4 series are exploratory; results should be validated on larger, more diverse corpora such as those surveyed in recent deep learning and foundation-model reviews [42, 48].
2. **Linear SARIMA assumption**: the diagnostic framework inherits the model family's limitations on high-volatility and multi-seasonal series.
3. **Refitting is not guaranteed to succeed**: Bitcoin's persistent diagnostic failure shows that increasing SARIMA order cannot address all forms of mis-specification.
4. **Computational cost**: ≈200× slower than Prophet; unsuitable for real-time or large-scale applications.
5. **Univariate only**: no support for multivariate (VAR/VARMA) modelling, in contrast to some modern transformer-based and foundation-model forecasters [27, 30–34, 44, 49].

8 Conclusion and Future Work

We presented DGSAR, an open-source diagnostic-first SARIMA forecasting pipeline that integrates residual validation into the model selection loop. The key findings are:

1. The **diagnostic-gated refitting** mechanism is triggered on 30% of rolling origins, confirming it addresses a real, non-trivial failure mode. After gating, the average Ljung-Box pass rate is **85%** (Wilson 95% CI: [63%, 95%]) and the ACF exceedance pass rate is **100%**.
2. ARCH-LM testing reveals that Bitcoin and COVID-19 exhibit **100% volatility clustering**, correctly classifying them as requiring GARCH rather than higher-order SARIMA—a prescriptive diagnostic output absent from existing tools [6, 23].
3. On MASE (scale-free), DGSAR wins **5/5 versus Prophet**, 3/5 versus ETS, and 3/5 versus Theta, with the no-gating ablation achieving identical MASE, confirming that diagnostic gating adds no accuracy cost.
4. On the expanded M4 subset (41 series), LB pass rates reach **100% for Yearly** and 75–90% for Monthly/Quarterly, with overall MASE win rates of 75.6% versus Prophet.

The central contribution is methodological: a reproducible framework that pairs accuracy benchmarking with principled residual validation and automatic remediation, providing the statistical transparency that regulated forecasting applications require. It is complementary to, rather than a replacement for, ongoing advances in neural and foundation-model forecasting [26–35, 41, 42, 44–46, 48, 49].

8.1 Future Directions

... Several extensions are planned to address the current limitations:

1. **GARCH-ARIMA integration**: when ARCH-LM fails, automatically switch to a GARCH(1,1)-ARIMA model instead of escalating SARIMA order, extending the diagnostic-gated framework to handle variance mis-specification [6, 23].

2. **Structural break detection**: incorporating CUSUM tests for regime changes in series such as COVID-19 and Bitcoin, where non-stationarity cannot be addressed by order escalation alone.
3. **Neural forecaster diagnostics**: applying the LB + ARCH diagnostic pipeline to LSTM and Transformer residuals to assess whether deep learning and foundation-model forecasters [26–35, 41, 42, 44–46, 48, 49] produce white-noise residuals.
4. **Large-scale evaluation**: scaling the benchmark to the full M4 and M5 corpora and Monash Archive with all baseline comparisons to establish population-level diagnostic and accuracy statistics [7, 15, 16].

References

[1] Taylor, S.J., Letham, B.: Forecasting at scale. *The American Statistician* **72**(1), 37–45 (2018). https://doi.org/10.1080/00031305.2017.1380080

[2] Ljung, G.M., Box, G.E.P.: On a measure of lack of fit in time series models. *Biometrika* **65**(2), 297–303 (1978).

[3] Shapiro, S.S., Wilk, M.B.: An analysis of variance test for normality. *Biometrika* **52**(3/4), 591–611 (1965).

[4] Diebold, F.X., Mariano, R.S.: Comparing predictive accuracy. *Journal of Business & Economic Statistics* **13**(3), 253–263 (1995).

[5] Harvey, D., Leybourne, S., Newbold, P.: Testing the equality of prediction mean squared errors. *International Journal of Forecasting* **13**(2), 281–291 (1997).

[6] Engle, R.F.: Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica* **50**(4), 987–1007 (1982).

[7] Makridakis, S., Spiliotis, E., Assimakopoulos, V.: The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting* **36**(1), 54–74 (2020).

[8] Hyndman, R.J., Athanasopoulos, G.: *Forecasting: Principles and Practice*, 2nd edn. OTexts, Melbourne (2018). https://otexts.com/fpp2/

[9] Box, G.E.P., Jenkins, G.M., Reinsel, G.C., Ljung, G.M.: *Time Series Analysis: Forecasting and Control*, 5th edn. Wiley, Hoboken (2015).

[10] Smith, T.G. et al.: *pmdarima: ARIMA estimators for Python* (2017–). https://alkaline-ml.com/pmdarima

[11] Cleveland, R.B. et al.: STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics* **6**(1), 3–73 (1990).

[12] Künsch, H.R.: The jackknife and the bootstrap for general stationary observations. *Annals of Statistics* **17**(3), 1217–1241 (1989).

[13] Board of Governors of the Federal Reserve System: Supervisory Guidance on Model Risk Management (SR 11-7). Federal Reserve, Washington, D.C. (2011). https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm

[14] Li, S. et al.: Enhancing the locality and breaking the memory bottleneck of Transformer on time series forecasting. *NeurIPS* **32** (2019).

[15] Makridakis, S., Spiliotis, E., Assimakopoulos, V.: M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting* **38**(4), 1346–1364 (2022). https://doi.org/10.1016/j.ijforecast.2021.11.013

[16] Godahewa, R., Bergmeir, C., Webb, G.I., Hyndman, R.J., Montero-Manso, P.: Monash Time Series Forecasting Archive. *arXiv preprint* arXiv:2105.06643 (2021). https://doi.org/10.48550/arXiv.2105.06643

[17] Hyndman, R.J., Khandakar, Y.: Automatic time series forecasting: The `forecast` package for R. *Journal of Statistical Software* **27**(3), 1–22 (2008). https://www.jstatsoft.org/article/view/v027i03

[18] Akaike, H.: A new look at the statistical model identification. *IEEE Transactions on Automatic Control* **19**(6), 716–723 (1974). https://doi.org/10.1109/TAC.1974.1100705

[19] Schwarz, G.: Estimating the dimension of a model. *Annals of Statistics* **6**(2), 461–464 (1978). https://doi.org/10.1214/aos/1176344136

[20] Tashman, L.J.: Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting* **16**(4), 437–450 (2000). https://doi.org/10.1016/S0169-2070(00)00065-0

[21] Bergmeir, C., Benítez, J.M.: On the use of cross-validation for time series predictor evaluation. *Information Sciences* **191**, 192–213 (2012). https://doi.org/10.1016/j.ins.2011.12.028

[22] Politis, D.N., Romano, J.P.: The stationary bootstrap. *Journal of the American Statistical Association* **89**(428), 1303–1313 (1994). https://doi.org/10.1080/01621459.1994.10476870

[23] Bollerslev, T.: Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics* **31**(3), 307–327 (1986). https://doi.org/10.1016/0304-4076(86)90063-1

[24] Gardner, E.S.: Exponential smoothing: The state of the art—Part II. *International Journal of Forecasting* **22**(4), 637–666 (2006). https://doi.org/10.1016/j.

ijforecast.2006.03.005

[25] Assimakopoulos, V., Nikolopoulos, K.: The theta model: A decomposition approach to forecasting. *International Journal of Forecasting* **16**(4), 521–530 (2000). https://doi.org/10.1016/S0169-2070(00)00066-2

[26] Salinas, D., Flunkert, V., Gasthaus, J., Januschowski, T.: DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting* **36**(3), 1181–1191 (2020). https://doi.org/10.1016/j.ijforecast.2019.07.001

[27] Lim, B. et al.: Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting* **37**(4), 1748–1764 (2021). https://doi.org/10.1016/j.ijforecast.2021.03.012

[28] Oreshkin, B.N., Carpov, D., Chapados, N., Bengio, Y.: N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. *arXiv preprint* arXiv:1905.10437 (2020). https://doi.org/10.48550/arXiv.1905.10437

[29] Challu, C. et al.: N-HiTS: Neural Hierarchical Interpolation for Time Series Forecasting. *arXiv preprint* arXiv:2201.12886 (2022). https://doi.org/10.48550/arXiv.2201.12886

[30] Zhou, H. et al.: Informer: Beyond efficient Transformer for long sequence time-series forecasting. *Proceedings of AAAI* **35**(12), 11106–11115 (2021). https://doi.org/10.1609/aaai.v35i12.17325

[31] Wu, H. et al.: Autoformer: Decomposition Transformers with Auto-Correlation for long-term series forecasting. *Advances in Neural Information Processing Systems* **34**, 22419–22430 (2021). https://papers.nips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html

[32] Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J.: A Time Series is Worth 64 Words: Long-term forecasting with Transformers. *International Conference on Learning Representations* (ICLR) (2023). https://doi.org/10.48550/arXiv.2211.14730

[33] Ansari, A.F. et al.: Chronos: Learning the Language of Time Series. *arXiv preprint* arXiv:2403.07815 (2024). https://arxiv.org/abs/2403.07815

[34] Das, A. et al.: A decoder-only foundation model for time-series forecasting. *Proceedings of ICML* (2024). https://research.google/blog/a-decoder-only-foundation-model-for-time-series-forecasting/

[35] Jin, M. et al.: Time-LLM: Time series forecasting by reprogramming large language models. *arXiv preprint* arXiv:2310.01728 (2023). https://arxiv.org/abs/2310.01728

[36] Shchur, O. et al.: AutoGluon–TimeSeries: AutoML for probabilistic time series forecasting. *Proceedings of the AutoML Conference* **224**, 1–26 (2023). https://proceedings.mlr.press/v224/shchur23a.html

[37] Alsharef, A. et al.: Review of ML and AutoML solutions to forecast time-series data. *Archives of Computational Methods in Engineering* **29**, 3707–3742 (2022). https://doi.org/10.1007/s11831-022-09765-0

[38] Hewamalage, H., Ackermann, K., Bergmeir, C.: Forecast evaluation for data scientists: Common pitfalls and best practices. *Data Mining and Knowledge Discovery* **37**(2), 788–832 (2023). https://doi.org/10.1007/s10618-022-00894-5

[39] Hassani, H., Yeganegi, M.R.: Selecting optimal lag order in Ljung–Box test. *Physica A: Statistical Mechanics and its Applications* **541**, 123700 (2020). https://doi.org/10.1016/j.physa.2019.123700

[40] Abbasimehr, H., Behboodi, A., Bahrini, A.: A novel hybrid model to forecast seasonal and chaotic time series. *Expert Systems with Applications* **239**, 122461 (2024). https://doi.org/10.1016/j.eswa.2023.122461

[41] Achour, S., Bouher, Y., Nguyen, D., Chesneau, N.: Foundation models for time series forecasting: Application in conformal prediction. *arXiv preprint* arXiv:2507.08858 (2025). https://doi.org/10.48550/arXiv.2507.08858

[42] Kottapalli, S.R.K. et al.: Foundation Models for Time Series: A Survey. *arXiv preprint* arXiv:2504.04011 (2025). https://arxiv.org/abs/2504.04011

[43] Yu, Y., Peeperkorn, J., De Smedt, J., De Weerdt, J.: Time Series Foundation Models for Process Model Forecasting. *arXiv preprint* arXiv:2512.07624 (2025). https://doi.org/10.48550/arXiv.2512.07624

[44] Fu, X., Li, Y., Papaioannou, G., Kim, Y.: Reverso: Efficient Time Series Foundation Models for Zero-shot Forecasting. *arXiv preprint* arXiv:2602.17634 (2026). https://doi.org/10.48550/arXiv.2602.17634

[45] Karaouli, N. et al.: How foundational are foundation models for time series forecasting? *arXiv preprint* arXiv:2510.00742 (2025). https://doi.org/10.48550/arXiv.2510.00742

[46] Rahimikia, E., Ni, H., Wang, W.: Re(Visiting) Time Series Foundation Models in Finance. *arXiv preprint* arXiv:2511.18578 (2025). https://doi.org/10.48550/arXiv.2511.18578

[47] European Central Bank: ECB guide to internal models. European Central Bank, Frankfurt am Main (2024). https://www.bankingsupervision.europa.eu/ecb/pub/pdf/ssm.supervisory_guides202402_internalmodels.en.pdf

[48] Kim, J. et al.: A comprehensive survey of deep learning for time series forecasting. *Artificial Intelligence Review* **58**, 216 (2025). https://doi.org/10.1007/s10462-025-11223-9

[49] Zhou, T. et al.: Inverted Transformers are effective for time series forecasting. *International Conference on Learning Representations* (ICLR) (2024). https://arxiv.org/abs/2310.06625